VULNERABILITY ASSESSMENT AND PENETRATION TESTING REPORT

FOR

https://hack-yourself-first.com/

FROM

SAUDA MOMIN

CONTENTS

SUMMARY

1.1 EXECUTIVE SUMMARY

INSTITUTE OF INFORMATION SECURITY (IIS) has assigned the task of vulnerability assessment and penetration testing (VAPT) on **https://hack-yourself-first.com/**

This VAPT was performed on 27th june 2025. The detailed report and our findings are described below.

1.2 OBJECTIVE

The objective of this test was to determine security vulnerabilities in the web server configuration and website running on the server. The tests were carried out assuming the identity of an attacker or with malicious intent. At the same time due care was taken not to harm the web server.

1.3 SCOPE

The scope of our work was limited to only **https://hack-yourself-first.com/** The vulnerabilities and our findings are described below.

TABULAR SUMMARY

The following tables summarize the vulnerability assessment of the server.

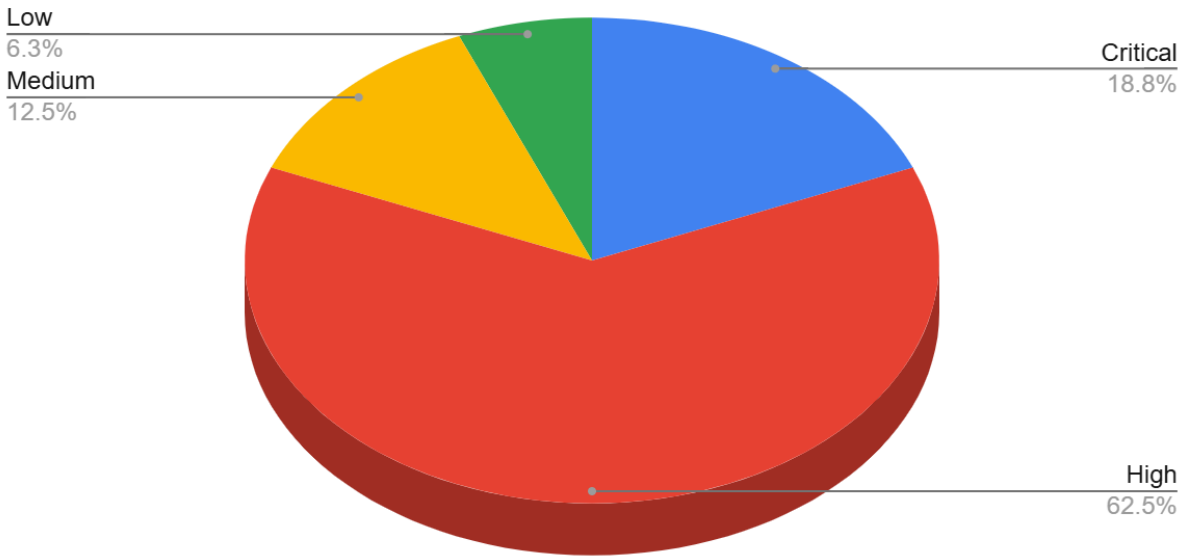
CATEGORY	DESCRIPTION
No of live host	1
No of vulnerabilities	16
No of critical vulnerabilities	3
No of high vulnerabilities	8
No of medium vulnerabilities	2
No of low vulnerabilities	1

Fig 2.1

SrNo.	Vulnerability Name	Severity	Ease of Exploitation
1	Passwords stored and displayed in plaintext	**Critical**	Moderate
2	AuthCookie accessible via JavaScript (Session Hijacking)	**Critical**	Moderate
3	Cookie manipulation – Missing Function Level Access Control	**Critical**	Very Easy
4	Stored Cross-Site Scripting (XSS)	**High**	Very Easy
5	IDOR in Supercar and User Profile Pages	**High**	Very Easy
6	Security Misconfiguration – SQL Error Message Disclosure	**High**	Moderate
7	Broken Authentication – Password reuse disclosure and weak password policy	**High**	Very Easy
8	Insufficient Logging and Monitoring – No detection for brute-force attempts	**High**	Easy
9	Insecure Design – Password sent via email	**High**	Easy
10	Using Components with Known Vulnerabilities – jQuery, Bootstrap, ASP.NET	**High**	Easy
11	Cryptographic Failure – Login over HTTP, missing HSTS	**High**	Easy
12	Missing/Misconfigured Security Headers	**High**	Easy
13	Broken Access Control – Direct access to `/secret/users`	**High**	Easy
14	Sensitive Data Exposure – Autocomplete Enabled on Login Form	**Medium**	Easy
15	Information Disclosure via `robot	**Medium**	Very easy
16	SQL Injection	**Low**	Easy

GRAPHICAL SUMMARY

The following pie chart graphically summaries the vulnerability assessment



3.3 – Pie chart showing the percentage distribution of vulnerabilities by severity

1. Passwords are stored and displayed in plaintext

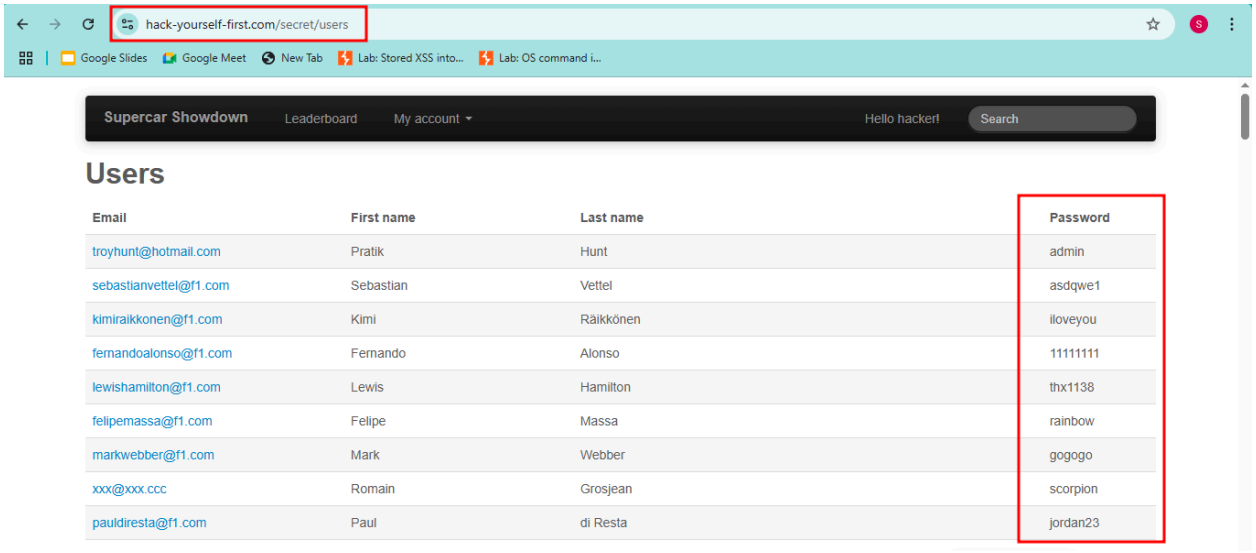
Affect on

Infected URL	https://hack-yourself-first.com/secret/users
Infected Parameter	User Passwords (fetched from backend storage)
Parameter type	Stored Credentials (Server-side variable / Database field)
Attack vector	Attacker gains access to the admin panel or database through common web exploits (e.g., SQL Injection, Broken Access Control, IDOR), and directly views **plaintext passwords** without needing decryption or brute-force effort

- Severity Level: Critical
- Ease of Exploitation: Moderate

Analysis

The application stores user passwords in plaintext, violating basic security practices. An attacker who gains access to the backend or admin interface can view all user credentials directly, without needing to crack them. This constitutes a complete cryptographic failure and makes the system highly vulnerable to secondary attacks like credential reuse. The absence of hashing or encryption represents a fundamental misconfiguration in secure data handling, warranting immediate remediation.



Impact:

- Full compromise of user accounts since passwords are stored and shown in plaintext
- Enables credential stuffing attacks on other platforms if users reuse passwords
- Breach of compliance regulations such as GDPR, PCI-DSS, HIPAA
- Damage to user trust and organizational reputation

Root Cause:

- Passwords are stored without hashing or encryption
- No salting or secure credential storage mechanisms are implemented
- Credentials are displayed in the admin panel or retrievable via API responses

Mitigation:

- Immediately replace plaintext storage with secure password hashing using bcrypt, scrypt, or Argon2
- Use a unique salt for each password
- Do not log, display, or expose hashed passwords via UI or API
- Force a password reset for all existing users if passwords were exposed
- Perform a full security audit of user authentication and data handling mechanisms

2. AuthCookie accessible via JavaScript (Session Hijacking)

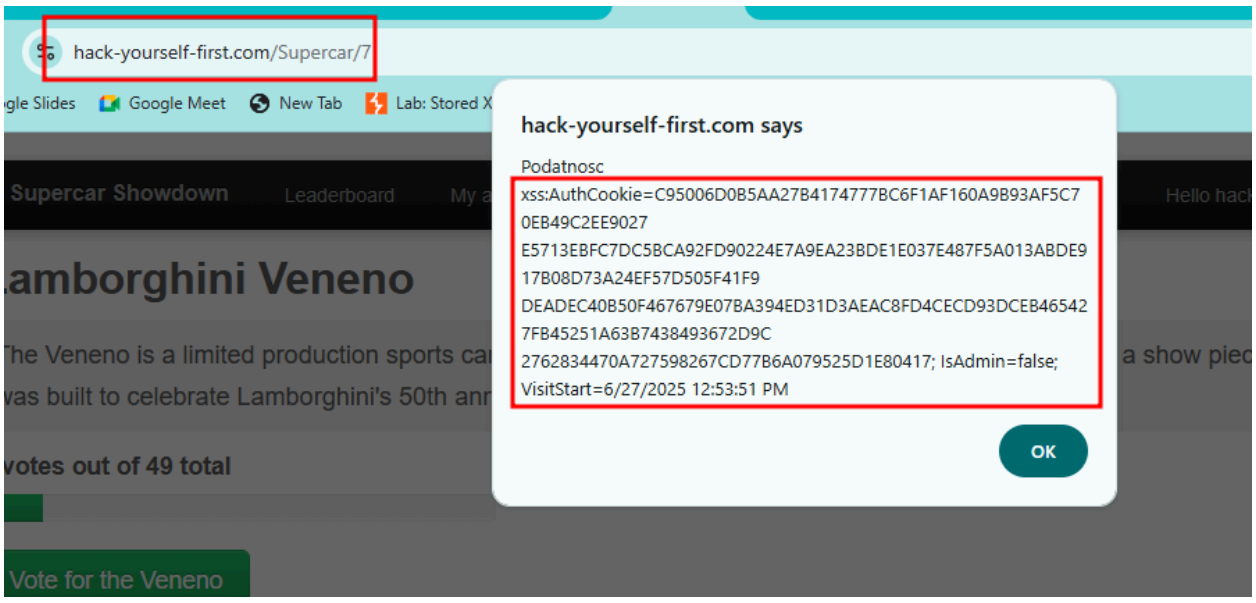
Affect on

Infected URL	https://hack-yourself-first.com/Supercar/7
Infected Parameter	AuthCookie
Parameter type	HTTP Cookie
Attack vector	The attacker uses a stored XSS to steal the victim's AuthCookie, then sets it in their own browser to access authenticated pages without logging in.

- Severity Level: Critical
- Ease of Exploitation: Moderate

Analysis

The application stores the AuthCookie in a way that allows access via JavaScript, indicating the HttpOnly flag is not set. This exposes the authentication token to client-side attacks such as stored XSS. An attacker can **capture the cookie value and reuse it in a separate session** to impersonate the user, demonstrating a critical identification and authentication failure that allows session hijacking without requiring credentials.



Impact

- Unauthorized access to user accounts through session hijacking
- Compromise of sensitive user data
- Bypass of login and authentication mechanisms
- Potential for privilege escalation if admin tokens are reused
- Violation of session confidentiality and integrity

Root Cause

- Session token (AuthCookie) accessible to JavaScript
- Missing HttpOnly flag on authentication cookie
- No session binding to user attributes (IP address, User-Agent, etc.)
- Authentication mechanism relies solely on reusable cookies

Mitigation

- Set HttpOnly, Secure, and SameSite=Strict flags on all auth cookies
- Implement short-lived session tokens with automatic expiration
- Invalidate sessions on logout or inactivity
- Bind session tokens to IP address or browser fingerprint
- Monitor for concurrent sessions from different locations
- Implement multi-factor authentication for critical actions

3. Cookie manipulation – Missing Function Level Access Control

Affect on

Infected URL	https://hack-yourself-first.com/Admin
Infected Parameter	isAdmin cookie (manually added via browser DevTools)
Parameter type	HTTP Cookie
Attack vector	Using DevTools, manually added a new cookie

Severity Level: Critical

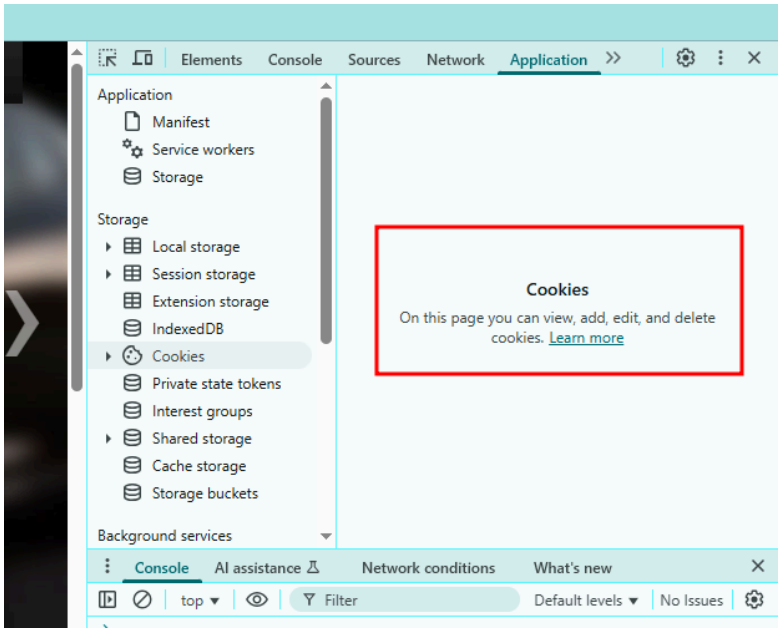
- Because it allows unauthorized privilege escalation and access to admin-only functionalities without authentication or validation.
- The impact affects confidentiality, integrity, and availability of the application and user data.

Ease of Exploitation: Very Easy

- No special tools required — the attack can be performed using browser Developer Tools by simply adding or editing a cookie.
- No authentication bypass or advanced techniques needed.
- Can be executed by any low-privileged user with basic browser knowledge.

Analysis

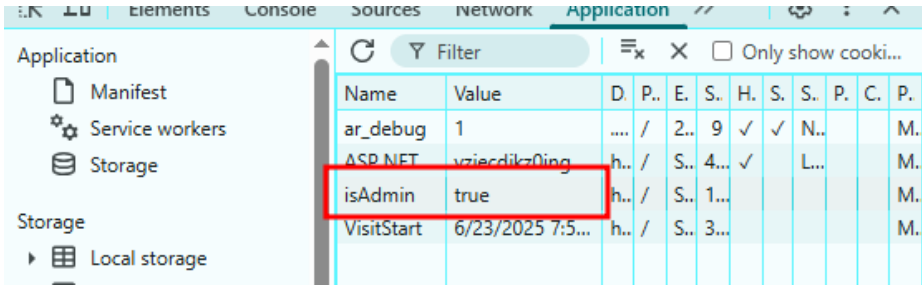
While testing the website, I observed that cookies can be **added, modified, and deleted** directly from the browser using DevTools.



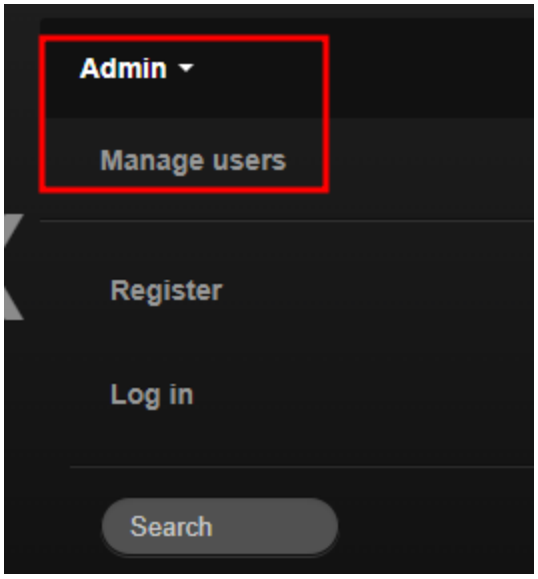
I was able to edit cookie values such as user roles, and the application accepted the changes without validation.

I added a new cookie with:

- Name: isAdmin
- Value: true



Then I refreshed the page. The application immediately **granted admin-level access**, showing admin features and pages that were previously restricted.



This confirms a **severe broken authentication vulnerability**, where the user's role or privilege level is controlled entirely through **client-side cookies** and not verified securely on the server side.

Impact

- Allows any user to escalate privileges and access admin-only pages or features.
- Exposes sensitive functionality and data intended for administrative users.
- Could lead to further exploitation, such as user data leakage, application misconfiguration, or complete system compromise.
- Violates secure access control principles and regulatory compliance requirements.

Root Cause

- The application relies on a **client-side indicator** (cookie) to manage user roles.
- No **server-side validation** is performed to confirm if a user is authorized to access the /Admin endpoint.
- Function-level access control checks are missing or improperly implemented.

Mitigations

- Do not trust client-side data to determine user roles or privileges.
- Store and manage access levels securely on the server side, tied to authenticated sessions.
- Remove the isAdmin cookie from client responses entirely.
- Implement proper role-based access control (RBAC) on the backend for all protected endpoints.

4. Stored Cross-Site Scripting (XSS)

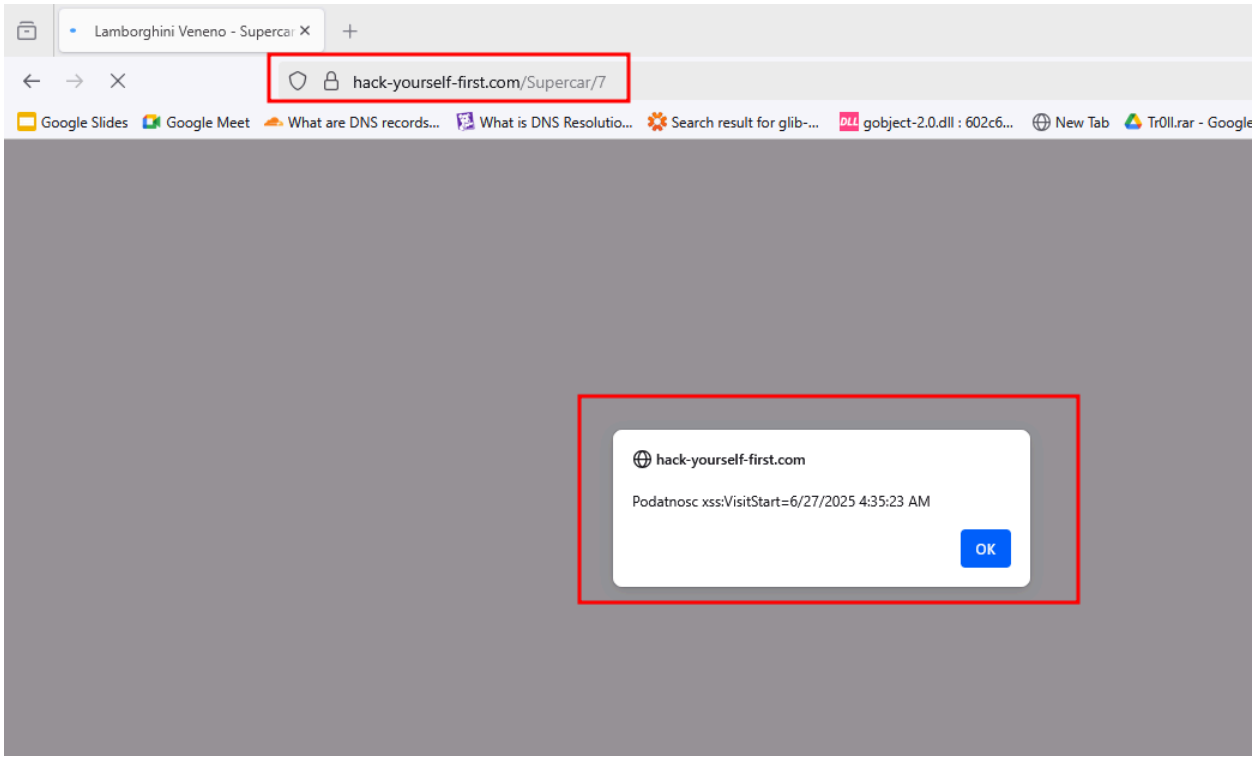
Affect on

Infected URL	https://hack-yourself-first.com/Supercar/7
Infected Parameter	No input was provided during the test. The payload was **already stored** on the server.
Parameter type	**stored server-side**, not passed via GET or POST parameters.
Attack vector	Stored JavaScript payload on the page executes automatically upon page load

- Severity Level:- High
- Ease of Exploitation :- **Very Easy** — The payload is already present and triggers automatically without any user input.

Analysis

While exploring the lab, I navigated to a page where a pop-up alert automatically appeared without any user interaction. The alert box displays session-related data:



The message is not the result of any user input or interaction, indicating the payload was stored previously, likely in a database field like "description", "comments", or similar.

While exploring more, I visited a supercar page where a JavaScript alert box with the number "1" appears automatically. I did not enter any input or inject any script myself.

This also confirms that a Stored Cross-Site Scripting (XSS) vulnerability exists on the page because the JavaScript payload was already saved on the server and executed every time the page was loaded.

Impact

- Stored XSS affects all users who visit the infected page.
- Can be used to steal session cookies, impersonate users, or carry out unauthorized actions.
- In this case, session information was already being exposed, indicating potential for further exploitation (such as extracting authentication tokens or privilege flags like IsAdmin).
- The vulnerability allows persistent execution of malicious scripts hosted on the server.

Root Cause

- User input (possibly from an admin or another user) was stored on the server without proper input validation or sanitization.
- Stored data was rendered into the HTML without escaping or encoding.
- This led to direct insertion of executable JavaScript code into the DOM.

Mitigation

- Apply proper output encoding on all dynamic data before rendering it to the page.
- Sanitize and validate all user inputs before storing them in the database.
- Implement a strong Content Security Policy (CSP) to restrict the execution of inline scripts and only allow trusted sources.
- Use secure templating engines that escape output by default.
- Disable or sanitize the use of potentially dangerous HTML tags and attributes in user input.
- Perform regular application security assessments focusing on input/output handling mechanisms.

5. IDOR in super car page (Broken Access Control)

Affect on

Infected URL	https://hack-yourself-first.com/Supercar/1 https://hack-yourself-first.com/Supercar/2 https://hack-yourself-first.com/Supercar/3
Infected Parameter	Object identifier in the URL path
Parameter type	Path Parameter
Attack vector	Changing the numeric ID in the URL from one value to another (e.g., /Supercar/1 → /Supercar/2) directly loaded a new resource without any authentication or access control check.

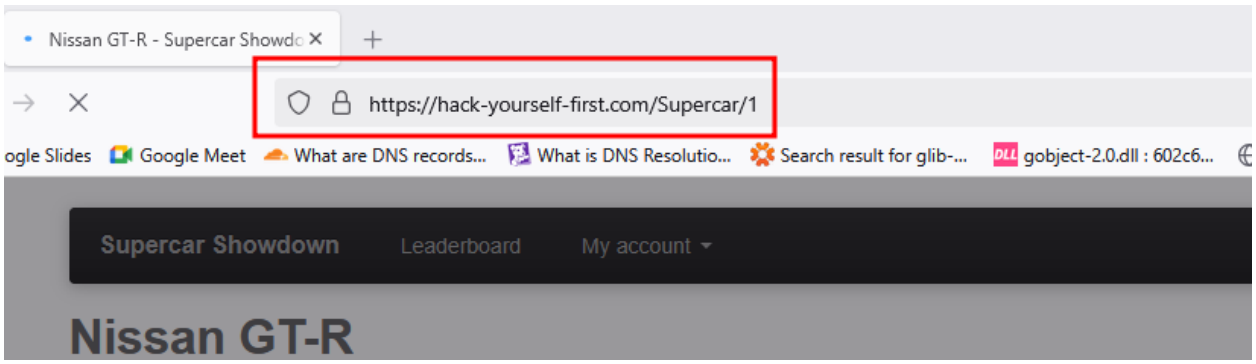
Affect on

Infected URL	https://hack-yourself-first.com/User/Profile/147
Infected Parameter	UserID (the numeric value at the end of the URL path)
Parameter type	Path Parameter
Attack vector	Changing the numeric user ID in the profile URL path allowed unauthorized access to other users' profile data.

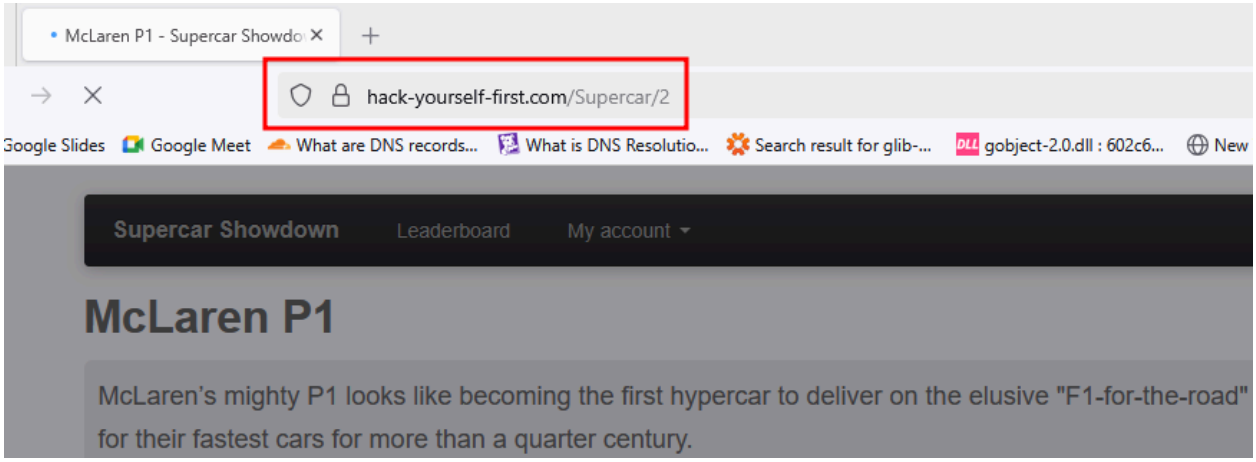
- Severity Level:- High
- Ease of Exploitation:- **Very Easy** — anyone with access to the site can manipulate numeric values in the URL and access different records without restrictions.

Analysis

While exploring the Supercar Showdown section on the website https://hack-yourself-first.com, I noticed that each car had its own page with a URL like this:



Out of curiosity, I changed the number at the end of the link from **1 to 2**, like this:



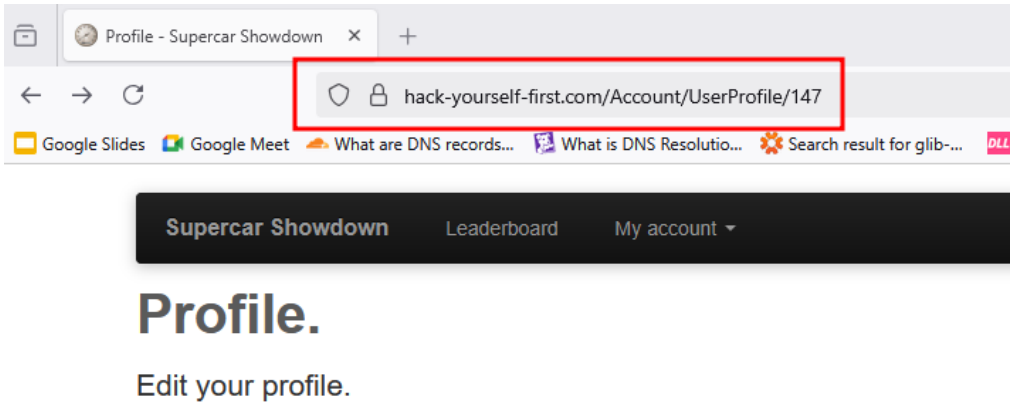
The website loaded a different page without any error. I tried more numbers and each one showed a new page. The website did not stop me or ask for any permission.

This shows that the site is using the number in the URL to find and display data, but it **doesn't check if the user is allowed to see that data**. This type of issue is called **IDOR (Insecure Direct Object Reference)**.

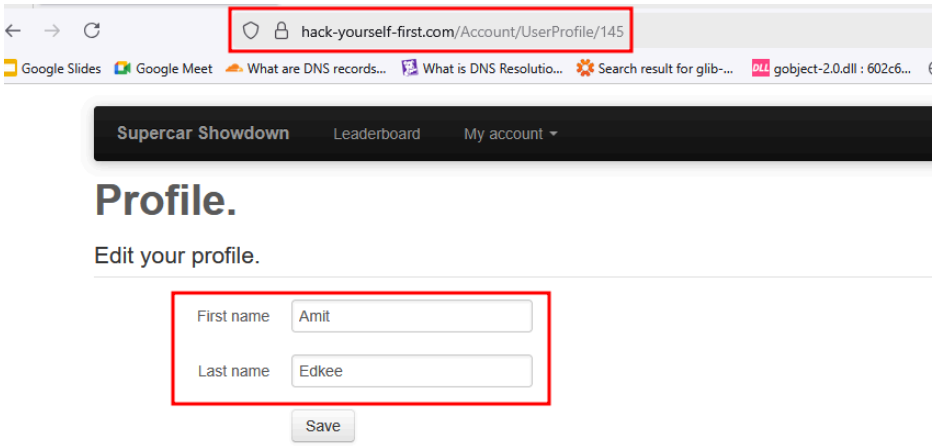
It means anyone can guess or change the number in the URL and see other data, even data that might not be meant for them. This can be risky, especially if it allows access to private or sensitive information.

B. IDOR (Broken Access Control) on user profile page

While browsing the website https://hack-yourself-first.com, I opened the Edit Profile page from the user account section. The URL looked like this:



This page allowed me to view and possibly edit my user profile. Just to test how secure it was, I changed the number in the URL from **147 to 145**, like this:



Surprisingly, a different profile page opened. The page showed personal information such as the **first name** and **last name** of that user. This shows that the website does **not check if a user is allowed to access a specific profile**. It simply loads whatever profile ID is entered in the URL.

Impact

- Allows unauthorized users to access data by modifying the object reference (ID) in the URL.
- If extended to other objects (e.g., users, orders, invoices, or profiles), could lead to:
    - Exposure of personal or confidential information
    - Unauthorized access to sensitive business data
    - Bypassing of role-based access controls

Root Cause

- The application uses direct object references in the URL (/Supercar/{id}).
- There is no access control validation to check if the user is authorized to access the requested object.
- The backend blindly trusts the client-supplied identifier and returns the associated resource.

Mitigation

- Implement proper access control checks on the server for every request to sensitive or private resources.
- Use indirect references (e.g., UUIDs or hashed values) instead of predictable numeric IDs.
- Do not rely on client-side logic to enforce access control.

6. Security Misconfigurations - – SQL Error Message Disclosure

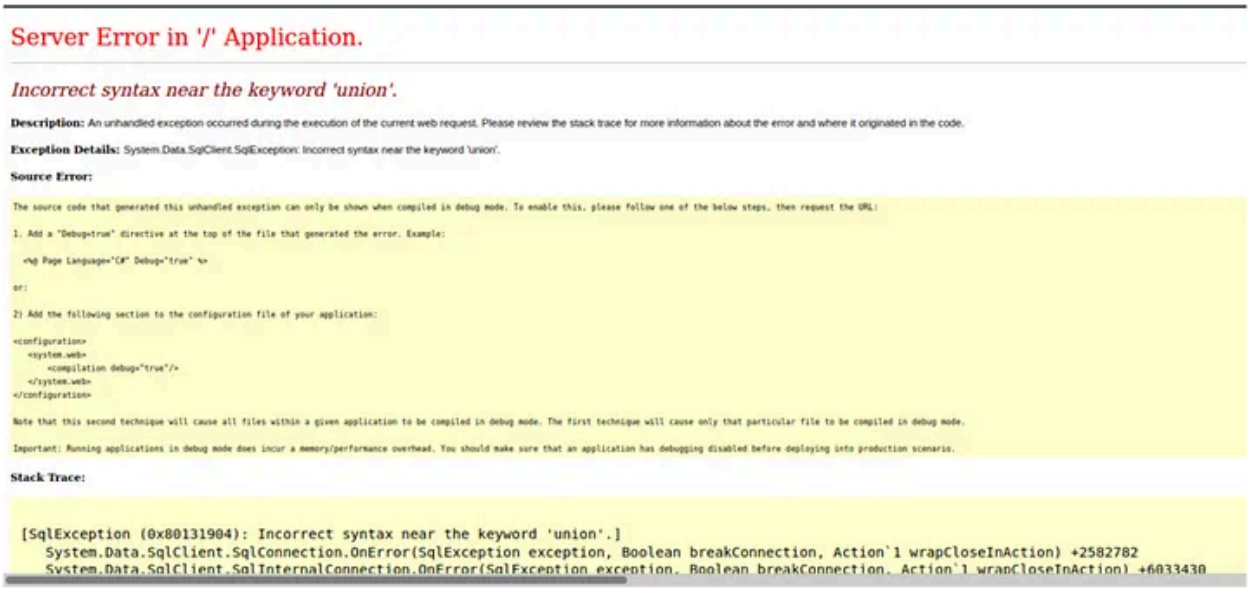
Affect on

Infected URL	https://hack-yourself-first.com/Products/Details/1
Infected Parameter	Query
Parameter type	Get
Attack vector	An attacker injected an invalid SQL statement into a parameter using the UNION keyword. The server returned a verbose error message showing the stack trace from the .NET SQLClient library, indicating that the application is directly passing user input into SQL queries without proper sanitization.

- Severity: High
- Ease of Exploitation: Moderate

Analysis

The error message clearly shows that the backend uses **Microsoft SQL Server** and **ADO.NET**. The message reveals internal class names and database interaction methods, such as ExecuteReader, RunExecuteReader, RunBehavior, etc. This level of error detail should never be exposed to the user as it aids in further exploitation.



Impact

- Confirms the presence of a SQL Injection vulnerability.
- Leaks backend logic and technology stack (SQL Server, .NET, etc.).
- Helps attackers craft more advanced payloads (e.g., blind SQLi, time-based SQLi).
- May allow attackers to read, modify, or delete database data**.**

Root Cause

- User input is directly embedded into SQL queries without validation, parameterization, or escaping.
- No input sanitization or use of prepared statements.
- Server is misconfigured to show detailed error messages to unauthenticated users.

Mitigations

- Use parameterized queries or ORM frameworks that prevent raw SQL injections.
- Implement input validation and sanitation on all user inputs.
- Disable detailed error messages in production environments; show only generic error messages to users.
- Log detailed errors securely on the server side, not exposed to end users.
- Conduct regular security testing (SAST, DAST) to catch SQLi and error-handling issues.

7. Broken authentication - – Password reuse disclosure and weak password policy

Affect on

Infected URL	https://hack-yourself-first.com/Account/Register
Infected Parameter	password field during user registration
Parameter type	Post
Attack vector	Entered a password during registration that had already been used by another user. The application responded with a message indicating that a specific user (fredfr@mail.com) had already used that password.

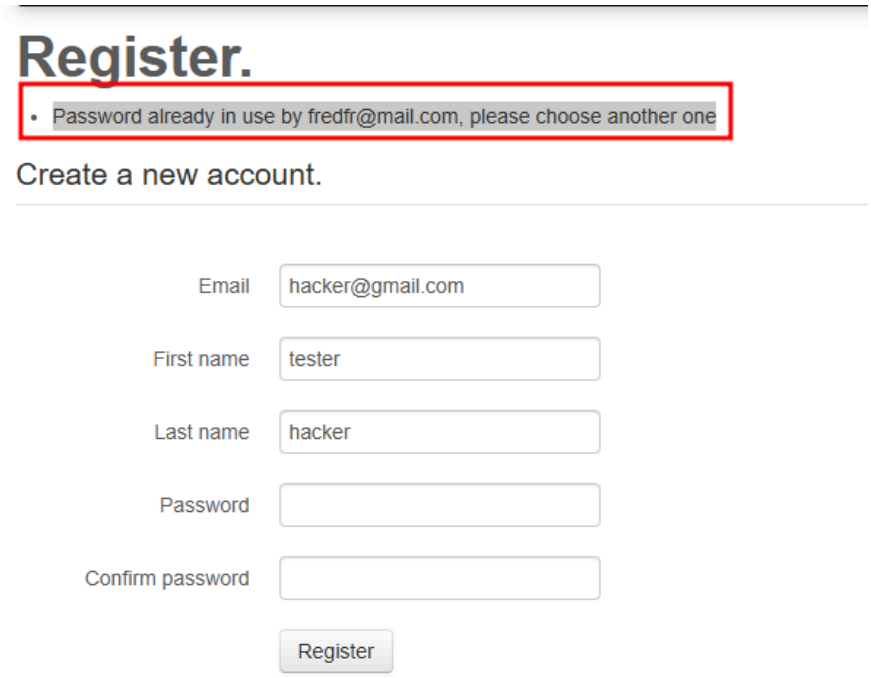
- Severity: High (Privilege escalation and credential disclosure make this a critical issue)
- Ease of Exploitation: Very Easy

Analysis

a. User credentials are not protected

While I was testing the registration feature, I found a broken authentication vulnerability in the **password input field**.

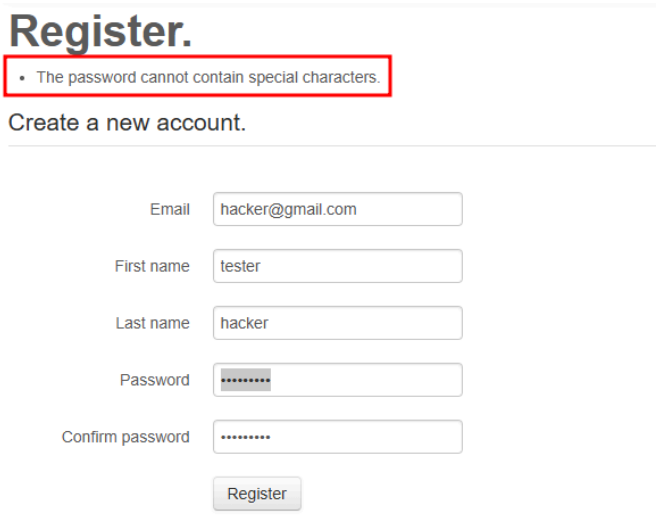
When I entered a password that was previously used by another user, the application responded with this message:



This response reveals that someone with the email **fredfr@mail.com** has already used this password. This is a serious security issue because the application should never show who is using a password.

b) Weak password

While testing the registration form, I also noticed that the application **does not allow special characters** (such as @, !, #, etc.) in the password field.



This is a bad practice because:

- Strong passwords require special characters to improve security.
- Restricting them encourages users to create weaker, more guessable passwords.
- It violates basic password hygiene standards and modern authentication recommendations.

Impact:

- Enables attackers to enumerate passwords and user identities during registration.
- Prevents users from creating strong passwords, reducing overall credential security.

Root Cause:

- Insecure business logic that discloses credential validation outcomes.
- Poor password policy design that rejects secure password formats.

Mitigation:

- Ensure password reuse checks are scoped per user, and display generic error messages without referencing other users.
- Update password validation rules to allow strong characters and follow OWASP/NIST guidelines.

8. Insufficient logging and monitoring - – No detection for brute-force attempts

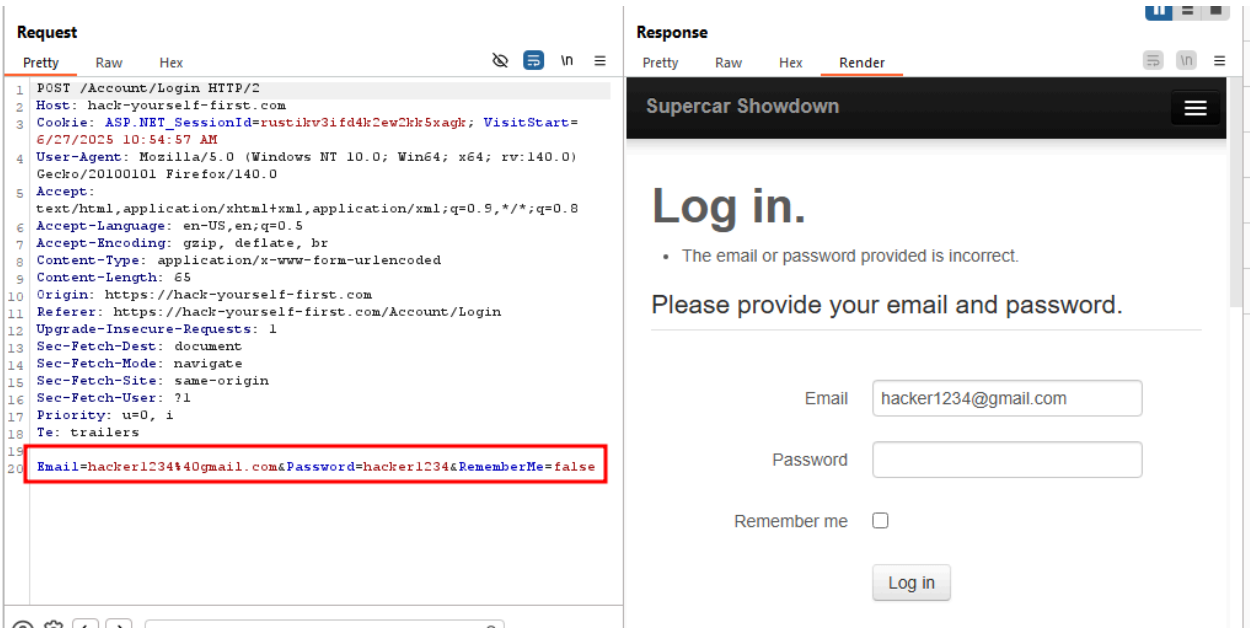
Affect on

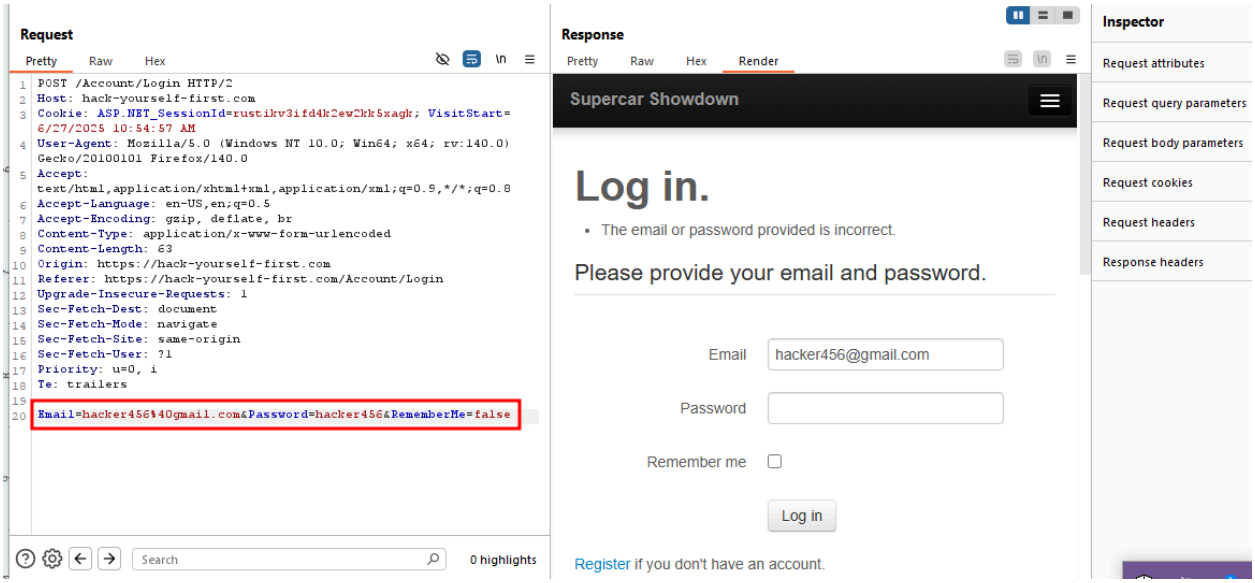
Infected URL	https://hack-yourself-first.com
Infected Parameter	Username & password (Login form)
Parameter type	Form Field Input (username, passwords)
Attack vector	Brute-force login with invalid credentials

- Severity: High
- Ease of Exploitation: Easy

Analysis

Multiple failed login attempts were performed on the /login endpoint of https://hack-yourself-first.com using Burp Suite. Despite entering incorrect credentials repeatedly, the application allowed unlimited attempts without triggering CAPTCHA, lockout, or any warning. The consistent HTTP 200 responses and lack of defensive behavior indicate that the application does not monitor or respond to brute-force activity. This confirms insufficient logging and monitoring of authentication-related events.





Impact:

- Attackers can perform malicious actions such as brute-force attacks, SQL injection, or privilege escalation without detection.
- Delayed or no detection of security breaches increases the time to respond and recover.
- Forensic analysis becomes impossible due to lack of audit trails.
- Violates compliance standards like PCI DSS, ISO 27001, and GDPR which require proper logging and monitoring.
- Chained exploitation becomes easier when earlier suspicious activities go unnoticed.

Root Cause:

- The application does not log critical events such as failed logins, suspicious input, or unauthorized access attempts.
- No alerting system is in place to notify administrators of potentially malicious activity.
- Lack of integration with centralized logging or SIEM systems.
- No rate limiting or CAPTCHA to detect or prevent automated attacks.
- Logs, if present, are not properly stored or monitored.

Mitigations:

- Log all authentication events including successful and failed login attempts, password reset requests, and account lockouts.
- Monitor for suspicious input patterns, such as SQL keywords or repeated abnormal requests.
- Store logs securely with restricted access and regular backups.
- Use a centralized logging system (e.g., ELK Stack, Splunk) integrated with alerting tools.
- Set up real-time alerts for high-risk activities like brute-force attacks or privilege escalation attempts.
- Regularly audit and review logs to identify anomalies and improve detection rules.
- Implement rate limiting and CAPTCHA on login and sensitive endpoints to deter automated attacks.
- Ensure that logging does not expose sensitive information such as passwords or session tokens.

9. Insecure design - – Password sent via email

Affect on

Infected URL	https://hack-yourself-first.com/Register.aspx
Infected Parameter	User credentials submitted during registration (e.g., email, password)
Parameter type	Form input
Attack vector	When a new user registers, their **plaintext password** is sent back to their email, exposing sensitive information through insecure transmission and long-term storage.

- Severity Level:- High
- Ease of Exploitation Easy

Analysis

Upon registering a new account, the application sends the user's password in **plaintext via email**. This violates secure design principles, as email is an inherently insecure medium (unencrypted, easily intercepted, and often stored for long periods in multiple locations).

Impact:

- Email interception can lead to immediate account compromise
- Long-term password exposure if email is compromised at any time
- Encourages poor user practices (e.g., password reuse across sites)
- Violates basic security principles (e.g., never store or transmit passwords in plaintext)

Root Cause:

- The system design includes sending stored or submitted passwords via email, instead of using secure password reset links or notifications.
- Indicates that passwords are likely being stored in **reversible or plaintext format** in the backend.

Mitigation:

- Never send passwords in plaintext via email or display them anywhere after creation.
- Use a secure, one-time password reset link if the user forgets their password.
- Hash passwords using a strong, one-way algorithm (e.g., bcrypt, Argon2).
- Educate users that passwords should be private and never shared—even by the system itself.
- Perform a design-level review to eliminate all patterns that assume password retrievability.

10. Using components with known vulnerabilities

Affect on

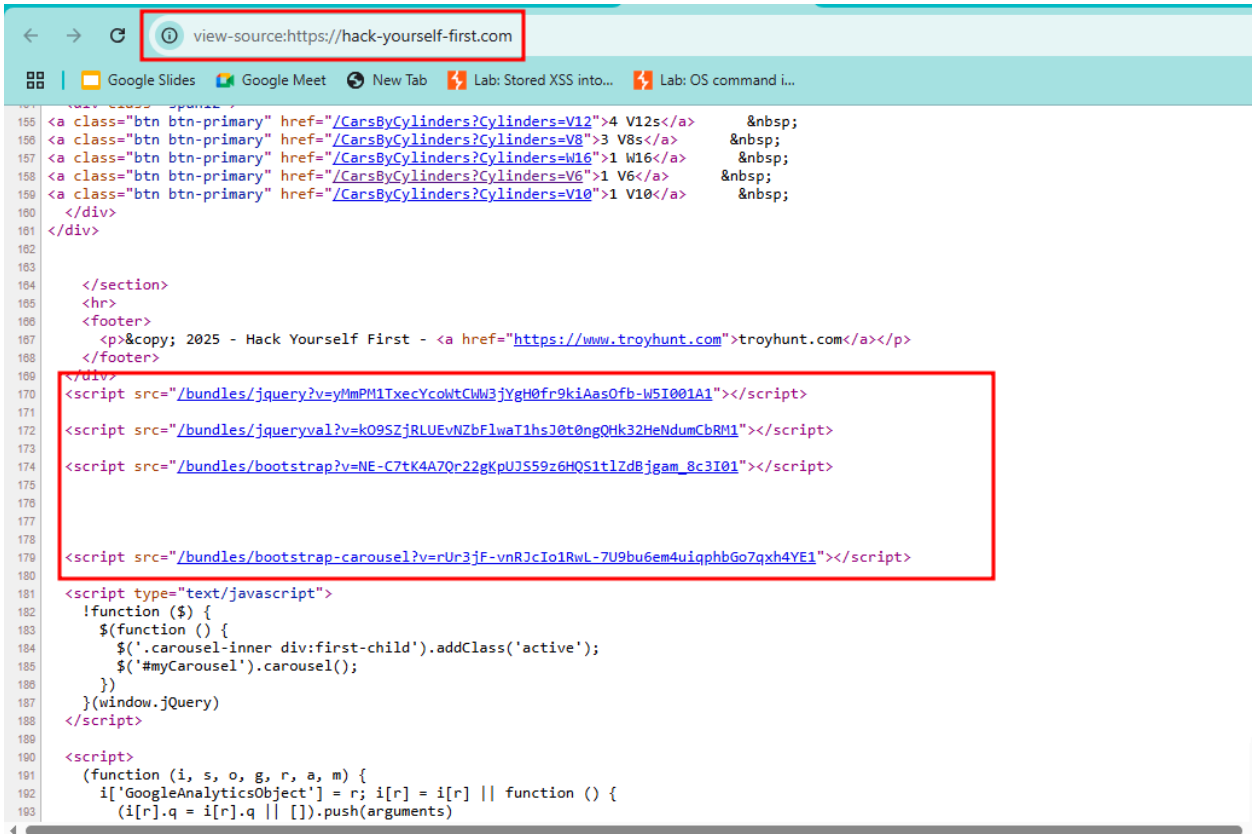
Infected URL	https://hack-yourself-first.com/
Infected Parameter	jQuery 1.10.2 (likely from /bundles/jquery) Bootstrap 3.1.1 (from /bundles/bootstrap) X-AspNet-Version, X-AspNetMvc-Version (Response Headers)
Parameter type	JavaScript and server-side component libraries Backend framework identifiers in HTTP response headers
Attack vector	Known public CVEs can be exploited based on the version of these components

- Severity Level:- High
- Ease of Exploitation Easy

Analysis

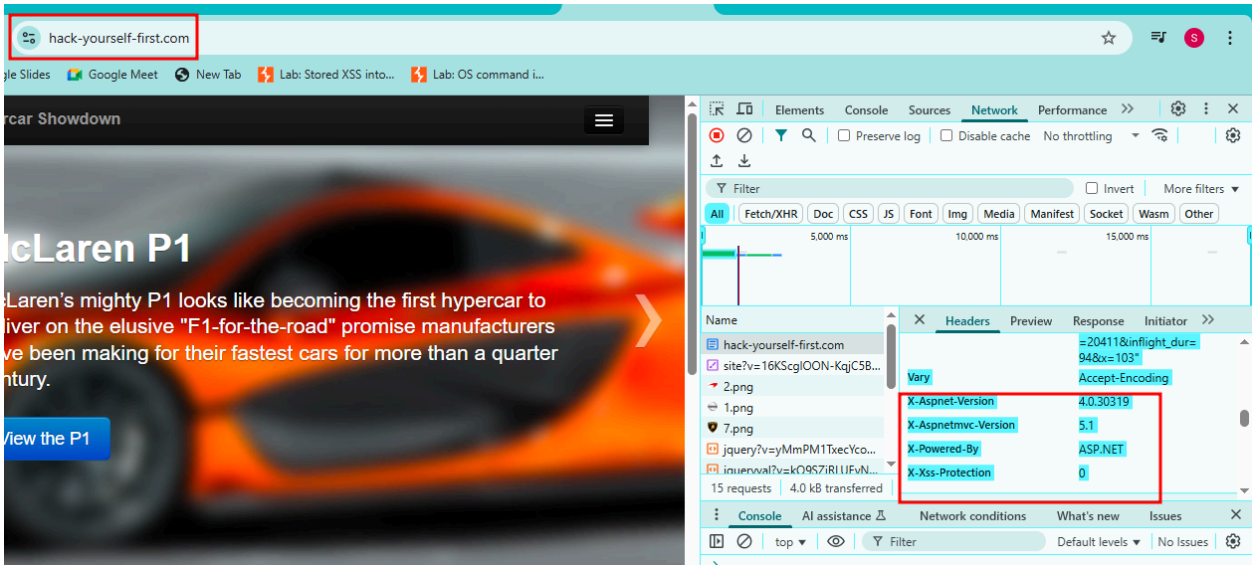
The application uses outdated libraries such as jQuery 1.10.2 and Bootstrap 3.1.1, which are known to contain publicly documented vulnerabilities like XSS. These components are included without version control or subresource integrity, exposing the application to client-side attacks. This indicates poor dependency management and lack of regular security updates.

The application reveals use of outdated backend components: ASP.NET 4.0.30319 and ASP.NET MVC 5.1, both of which have known vulnerabilities. Despite being behind Cloudflare, these exposed version headers allow attackers to identify and target known issues in the outdated frameworks, increasing the risk of exploitation.


view-source:https://hack-yourself-first.com
Google Slides
Google Meet
New Tab
Lab: Stored XSS into...
Lab: OS command i...
4 V12s
3 V8s
1 W16
1 V6
1 V10

</section>
<hr>
<footer>
© 2025 - Hack Yourself First - troyhunt.com
</footer>

<script src="/bundles/jquery?v=yMmPM1TxecYcoWtCWW3jYgH0fr9kiAasOfb-W5I001A1"></script>
<script src="/bundles/jqueryval?v=kO9SZjRLUEvNZbFlwaT1hsJ0t0ngQHk32HeNdumCbRM1"></script>
<script src="/bundles/bootstrap?v=NE-C7tK4A7Qr22gKpUJS59z6HQS1tlZdBjgam_8c3I01"></script>
<script src="/bundles/bootstrap-carousel?v=rUr3jF-vnRJcIo1RwL-7U9bu6em4uiqphbGo7qxh4YE1"></script>
<script type="text/javascript">
!function ($) {
$(function () {
$('.carousel-inner div:first-child').addClass('active');
$('#myCarousel').carousel();
})
}(window.jQuery)
</script>
<script>
(function (i, s, o, g, r, a, m) {
i['GoogleAnalyticsObject'] = r; i[r] = i[r] || function () {
(i[r].q = i[r].q || []).push(arguments)


hack-yourself-first.com
Google Meet
New Tab
Lab: Stored XSS into...
Lab: OS command i...
car Showdown
cLaren P1
Laren's mighty P1 looks like becoming the first hypercar to
iver on the elusive "F1-for-the-road" promise manufacturers
ve been making for their fastest cars for more than a quarter
ntury.
iew the P1
Elements
Console
Sources
Network
Performance
Preserve log
Disable cache
No throttling
Filter
Invert
More filters
All
Fetch/XHR
Doc
CSS
JS
Font
Img
Media
Manifest
Socket
Wasm
Other
5,000 ms
10,000 ms
15,000 ms
Name
Headers
Preview
Response
Initiator
hack-yourself-first.com
site?v=16KScgIOON-KqjC5B...
2.png
1.png
7.png
jquery?v=yMmPM1TxecYco...
15 requests
4.0 kB transferred
Vary
Accept-Encoding
X-Aspnet-Version
4.0.30319
X-Aspnetmvc-Version
5.1
X-Powered-By
ASP.NET
X-Xss-Protection
0
Console
AI assistance
Network conditions
What's new
Issues
top
Filter
Default levels
No Issues

Impact:

- Allows attackers to exploit publicly known bugs in outdated libraries (e.g., XSS)
- Attack complexity is low due to availability of public PoCs
- Poses risk to client-side integrity and server security

Root Cause:

- Use of old JavaScript libraries without regular updates
- No use of subresource integrity (SRI)
- No automated dependency checking in the development process

Mitigation:

- Upgrade to latest secure versions (e.g., jQuery 3.x, Bootstrap 5.x)
- Regularly audit libraries using tools like Retire.js, npm audit, or OWASP Dependency-Check
- Use Subresource Integrity (SRI) tags for external scripts
- Remove unused libraries and avoid unnecessary dependencies

11. Cryptographic Failure - – Login over HTTP, missing HSTS

Affect on

Infected URL	http://hack-yourself-first.com/Account/Login
Infected Parameter	Email, Password
Parameter type	Post
Attack vector	Credentials submitted over **unencrypted HTTP**. No automatic redirect to HTTPS. Allows attackers to intercept sensitive login data through Man-in-the-Middle (MitM) attacks on public or compromised networks.

- Severity Level:- High
- Ease of Exploitation Easy (Simple payload, no authentication required)

Analysis

The curl -I http://hack-yourself-first.com/ response shows HTTP/1.1 200 OK, which confirms that the server is serving content directly over unencrypted HTTP without redirecting to HTTPS. This means users can access the site insecurely, exposing them to Man-in-the-Middle (MitM) attacks.

The response also sets cookies (like ASP.NET_SessionId) without the Secure flag, meaning they could be transmitted over HTTP and intercepted. Together, these behaviors indicate a cryptographic failure, as sensitive data can be exposed due to lack of enforced encryption.

Impact:

- Allows attackers to intercept sensitive data such as login credentials or session cookies through Man-in-the-Middle (MitM) attacks.
- Users may unknowingly use the HTTP version, compromising the confidentiality and integrity of the connection.
- Enables downgrade attacks due to lack of enforced HTTPS.

Root Cause:

- The server responds to HTTP requests with a 200 OK status instead of redirecting to HTTPS.
- No HSTS header is configured to enforce secure transport on subsequent browser visits.
- Session cookies are not marked with the Secure flag, allowing transmission over insecure channels.

Mitigation:

- Configure the server to redirect all HTTP requests to HTTPS using permanent redirects (HTTP 301).
- Implement the Strict-Transport-Security header with appropriate directives to enforce HTTPS.
- Ensure all cookies containing session or sensitive data include the Secure and HttpOnly flags.
- Disable or block HTTP access entirely if HTTPS is supported.

12. Broken access control - – Direct access to /secret/users

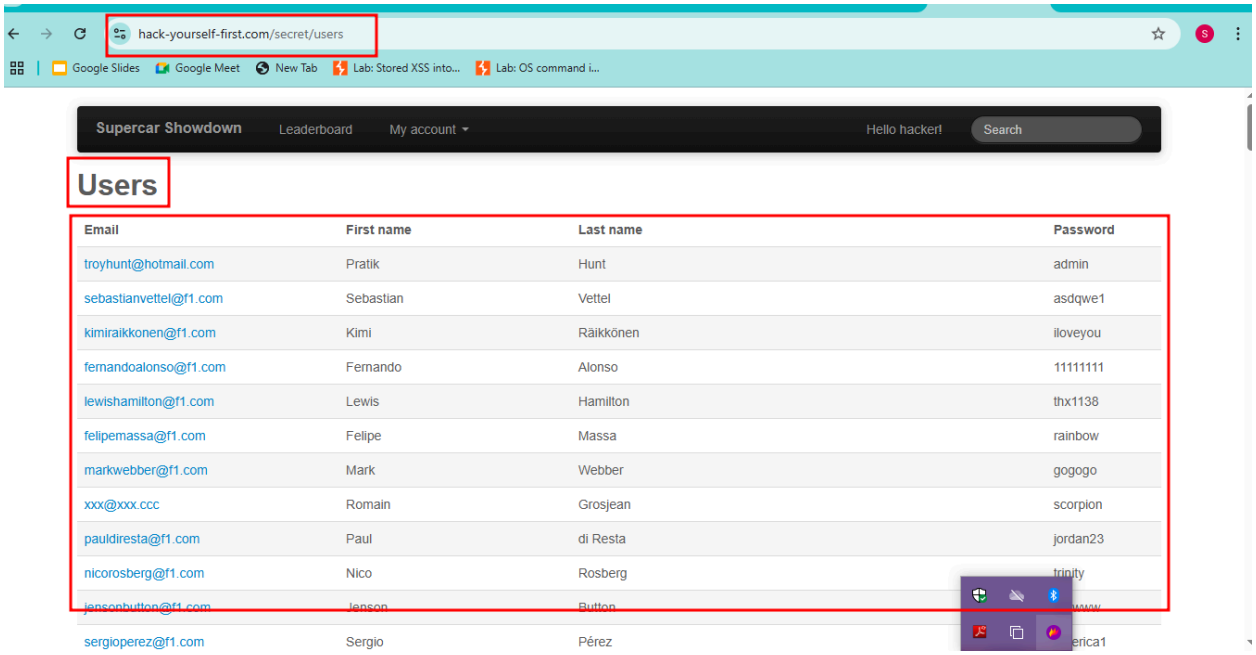
Affect on

Infected URL	https://hack-yourself-first.com/secret/users
Infected Parameter	Broken Access Control – Direct Access to Restricted Page
Parameter type	URL Path / Direct Object Reference
Attack vector	Directly navigating to /secret/users without authentication or authorization

- **Severity:** High
- **Ease of Exploitation:** Easy

Analysis

The /secret/users endpoint can be accessed by unauthenticated or unauthorized users directly through the browser. There is no server-side access control implemented to restrict this sensitive endpoint. This allows an attacker to enumerate or view user-related information that should be restricted to administrators or authenticated users.



Impact:

- Unauthorized access to internal or admin pages
- Leakage of sensitive user data (if present on page)
- Opens attack surface for user enumeration and privilege escalation

Root Cause:

- Absence of authentication or role validation on /secret/users route
- Reliance on security through obscurity (i.e., "hidden" URLs)

Mitigation:

- Apply authentication and authorization checks on all sensitive routes
- Implement Role-Based Access Control (RBAC)
- Deny access by default to internal pages and allow only verified roles
- Include access control checks in centralized middleware or controller filters

13. Sensitive data exposure - Autocomplete Enabled on Login Form

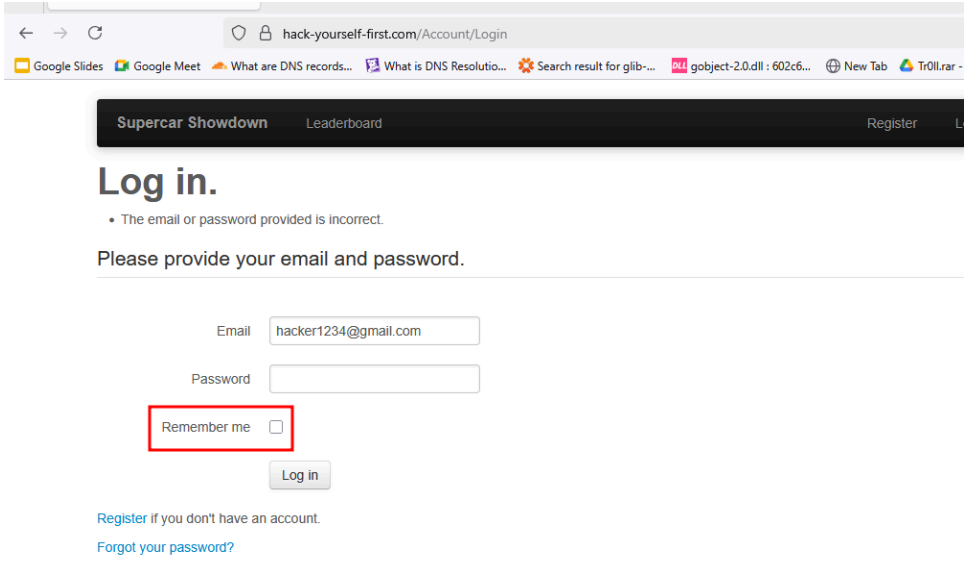
Affect on

Infected URL	https://hack-yourself-first.com/Account/Login
Infected Parameter	username and password input fields
Parameter type	Post
Attack vector	Do **not use autocomplete="off"**

- **Severity:** Medium
- **Ease of Exploitation:** Easy

Analysis

Many users accept the browser prompt to save passwords without understanding the security implications. If an attacker gains access to the user's machine or browser session (e.g., via XSS, physical access, or malware), the saved credentials can be extracted. This behavior violates secure design principles for authentication fields, especially for high-privilege areas like login, registration, and password reset.



```
<input data-val="true" data-val-required="The
Password field is required." id="Password" name=
"Password" type="password" />
<span class="field-validation-valid"
```

Impact:

- Sensitive user credentials (username and password) may be saved automatically by the browser.
- If an attacker gains access to the user's system (e.g., via physical access, session hijacking, or malware), stored credentials can be retrieved easily.
- In shared or public computer environments, subsequent users may gain access to previously saved credentials.
- Combined with other vulnerabilities like Cross-Site Scripting (XSS), credentials may be exfiltrated without user interaction.

Root Cause:

- The login and registration forms do not explicitly disable browser autofill functionality using the autocomplete="off" attribute.
- The application relies on default browser behavior, which assumes it is safe to prompt for password storage.
- There is a lack of secure form field handling and attention to input field-level security in authentication interfaces.

Mitigations:

- Add autocomplete="off" to all login and registration forms.
- Educate users not to save passwords on shared or public computers.
- Implement strong Content Security Policy (CSP) headers to limit the risk of XSS accessing stored values.
- Enforce secure password storage practices (e.g., salted hashing with bcrypt).
- Implement Multi-Factor Authentication (MFA) to reduce the risk of credential compromise.

14. Information Disclosure via robots.txt

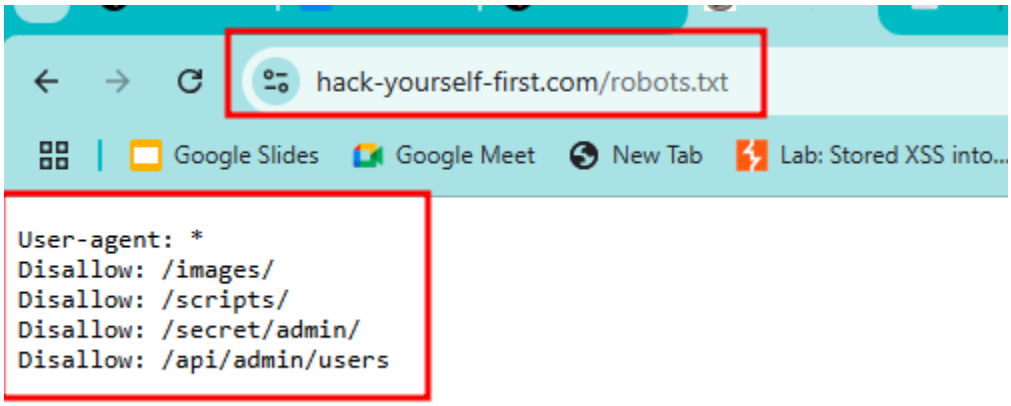
Affect on

Infected URL	https://hack-yourself-first.com/robots.txt
Infected Parameter	Not applicable (static file)
Parameter type	Resource/File (Configuration File)
Attack vector	Publicly accessible robots.txt file contains Disallow directives for sensitive and internal application paths.

- Severity Level:- Medium
- Ease of Exploitation:- Very easy

Analysis

The robots.txt file includes multiple Disallow directives pointing to sensitive or internal application paths. While this file is intended to guide search engine indexing, it is commonly accessed by attackers during reconnaissance. The presence of these disallowed paths effectively leaks information about application structure and potential attack targets.



Impact:

- Reveals presence and structure of hidden admin interfaces
- Enables targeted probing and enumeration of administrative APIs
- Assists in identifying application attack surface
- May expose unprotected or undocumented endpoints

Root Cause:

- robots.txt file is used incorrectly to hide sensitive paths rather than protect them
- No authentication or access control guarding listed paths

Mitigation:

- Do not list sensitive URLs in robots.txt; use authentication and authorization controls
- Use security by design, not by obscurity
- Ensure sensitive endpoints (like /api/admin/users) are properly protected regardless of visibility
- Regularly review public files (robots.txt, .env, .git, etc.) for unintended disclosures

15. SQL INJECTION

Affect on

Infected URL	https://hack-yourself-first.com/Supercar/Leaderboard' OR '1'='1
Infected Parameter	The payload was injected **into the URL path**, not as a parameter.
Parameter type	Path Parameter
Attack vector	Single quote with SQL logic injected into the endpoint path: Leaderboard' OR '1'='1

- Severity Level:- Low
- Ease of Exploitation Easy (Simple payload, no authentication required)

Analysis

The server returned the following error message:

This indicates that the application treated the injected value as part of the file path, not as a query or input field. Since the application does not parse user-controlled input from the path in this case, SQL injection was not triggered.

Impact

- None in this specific test, because the injection was performed in an incorrect context (URL path).
- However, improper handling of user input in URLs can still lead to route-based attacks or open the door for injection if interpreted differently by backend logic.
- Testing input in the correct query or form parameters is needed to evaluate actual SQLi exposure.

Root Cause

- This test did not result in a vulnerability due to:
    - No input vector was available at the tested endpoint (/Leaderboard)
    - The application does not parse the path string into SQL queries
- However, the application did not sanitize the input before using it in the routing mechanism, resulting in a server error response, which may disclose stack behavior in other contexts.

Mitigation

- Ensure routing logic is strict and does not process unexpected path values.
- Implement input validation and normalization on all incoming request paths and parameters.
- Return custom error pages instead of default .NET error messages to avoid leaking internal structure or configuration.
- Test other parts of the application (search boxes, filters, POST requests) for real SQL injection entry points.

CONCLUSION

The comprehensive vulnerability assessment of https://hack-yourself-first.com reveals that the application is highly vulnerable to a wide range of security flaws, many of which fall under the OWASP Top 10 categories. These include, Broken Access Control, Cryptographic Failures, Injection Attacks, Security Misconfigurations, Cross-Site Scripting (XSS), Insecure Design, and Use of Vulnerable Components.

Several critical issues were observed, such as:

- Plaintext storage and display of user passwords,
- Privilege escalation via client-side cookie manipulation,
- Lack of access control on sensitive endpoints, and
- Absence of basic protections like HTTPS enforcement, secure cookie attributes, and security headers.

These vulnerabilities expose the application to severe risks such as unauthorized access, data leakage, session hijacking, and complete account compromise. Moreover, the lack of secure coding practices, input validation, and output encoding reflects deeper systemic issues in the application's development and deployment lifecycle.